

Introduction to Computing System: course overview and lecture plan

Table of Contents

1	Course Overview	4
1.1	Overview	4
1.2	Why This Course Is Ideal for First-Semester Freshmen	5
1.3	What This Course Will Help You Understand.....	5
1.4	What You Will Build in This Course.....	6
1.5	No Prior Background Required.....	6
1.6	Course Description	7
1.7	Course Philosophy.....	7
1.8	Learning Objectives	7
1.9	Prerequisites	7
1.10	Tools and Resources.....	8
1.11	Skills Students Will Develop	10
1.12	How This Course Fits into the CS Curriculum	10
1.13	Course Structure at a Glance.....	11
1.14	What Is Expected from Students	11
2	Lectures (Weekly Plan).....	11
3	Projects.....	12
4	Assessment & Grading Breakdown.....	14
4.1	Attendance & Participation Policy (20%).....	14
4.1.1	General Notes	15
5	Weekly Course Breakdown	15
5.1	Week 1 – Lecture 0: Course Introduction & Computing Abstractions.....	15
5.1.1	Lecture Purpose	15
5.1.2	Teaching Goals (Instructor).....	15
5.1.3	Student Learning Outcomes	15
5.1.4	Key Concepts.....	15
5.1.5	Tools Introduction	16
5.1.6	Activities.....	16
5.1.7	Optional Project 0 (Highly Recommended)	17
5.1.8	Common Misconceptions (Address Explicitly)	17
5.1.9	End-of-Lecture Summary	17
5.1.10	Preparation for Next Lecture	17
5.2	Week 2 – Lecture 1 (Chapter 1): Boolean Logic	17
5.2.1	Lecture Purpose	17
5.2.2	Teaching Goals (Instructor).....	17
5.2.3	Student Learning Outcomes	17
5.2.4	Key Concepts.....	18
5.2.5	Project 1: Elementary Logic Gates	18
5.2.6	Project Focus	19
5.2.7	Tools Used.....	19
5.2.8	In-Class Activities	20
5.2.9	Common Student Difficulties	20
5.2.10	End-of-Lecture Summary	20
5.2.11	Preparation for Next Lecture	20
5.2.12	Resources.....	20
5.3	Week 3 – Lecture 2 (Chapter 2): Boolean Arithmetic	21
5.3.1	Lecture Purpose	21
5.3.2	Teaching Goals (Instructor).....	21
5.3.3	Student Learning Outcomes	21
5.3.4	Key Concepts.....	22
5.3.5	Project 2: Arithmetic Logic Unit.....	22
5.3.6	In-Class Activities	23

5.3.7	Common Student Difficulties	23
5.3.8	Preparation for Next Lecture	23
5.3.9	Resources	23
5.4	Week 4 – Lecture 3 (Chapter 3): Sequential Logic: Registers and Memory	24
5.4.1	Lecture Purpose	24
5.4.2	Teaching Goals (Instructor).....	24
5.4.3	Student Learning Outcomes	24
5.4.4	Key Concepts.....	25
5.4.5	Project 3: Memory	25
5.4.6	In-Class Activities	26
5.4.7	Common Student Difficulties	26
5.4.8	End-of-Lecture Summary	26
5.4.9	Preparation for Next Lecture	26
5.5	Week 5 – Lecture 4 (Chapter 4): Machine Language	27
5.5.1	Lecture Purpose	27
5.5.2	Teaching Goals (Instructor).....	27
5.5.3	Student Learning Outcomes	27
5.5.4	Key Concepts.....	27
5.5.5	Project 4: Machine Language Programming	28
5.5.6	Project Goal	28
5.5.7	What Students Do	28
5.5.8	Required Programs	29
5.5.9	Tools Used	30
5.5.10	Common Student Difficulties	30
5.5.11	End-of-Project Summary	31
5.5.12	Preparation for Next Lecture	31
5.6	Week 6 – Lecture 5 (Chapter 5): Computer Architecture	31
5.6.1	Lecture Purpose	31
5.6.2	Teaching Goals (Instructor).....	31
5.6.3	Student Learning Outcomes	31
5.6.4	Key Concepts.....	31
5.6.5	Project 5: Computer	33
5.6.6	In-Class Activities	33
5.6.7	Common Student Difficulties	33
5.6.8	End-of-Lecture Summary	34
5.6.9	Preparation for Next Lecture	34
5.7	Week 7 – Lecture 6 (Chapter 6): Assembler	34
5.7.1	Lecture Purpose	34
5.7.2	Teaching Goals (Instructor).....	34
5.7.3	Student Learning Outcomes	34
5.7.4	Key Concepts.....	34
5.7.5	Project 6: Assembler.....	36
5.7.6	Common Student Difficulties	37
5.7.7	End-of-Lecture Summary	37
5.7.8	Preparation for Next Lecture	37
5.8	Week 9 – Lecture 7 (Chapter 7): Virtual Machine I: Stack Arithmetic	38
5.8.1	Lecture Purpose	38
5.8.2	Teaching Goals (Instructor).....	38
5.8.3	Student Learning Outcomes	38
5.8.4	Key Concepts.....	38
5.8.5	Project 7: VM Translator (Part I)	39
5.8.6	Common Student Difficulties	41
5.8.7	End-of-Lecture Summary	41
5.8.8	Preparation for Next Lecture	41
5.9	Week 10 – Lecture 8 (Chapter 8): Virtual Machine II: Program Control	41
5.9.1	Lecture Purpose	41

5.9.2	Teaching Goals (Instructor).....	41
5.9.3	Student Learning Outcomes	42
5.9.4	Key Concepts.....	42
5.9.5	Project 8: VM Translator (Part II)	43
5.9.6	Common Student Difficulties	45
5.9.7	End-of-Lecture Summary	45
5.9.8	Preparation for Next Lecture	45
5.10	Week 11 – Lecture 9 (Chapter 9): High-Level Language.....	45
5.10.1	Lecture Purpose	45
5.10.2	Teaching Goals (Instructor).....	46
5.10.3	Student Learning Outcomes	46
5.10.4	Key Concepts.....	46
5.10.5	Project 9: High-Level OOP Design Exercise	49
5.10.6	Common Student Difficulties	51
5.10.7	End-of-Lecture Summary	51
5.10.8	Preparation for Next Lecture	51
5.11	Week 12 – Lecture 10 (Chapter 10): Compiler I: Syntax & Structure.....	51
5.11.1	Lecture Purpose	51
5.11.2	Teaching Goals (Instructor).....	51
5.11.3	Student Learning Outcomes	52
5.11.4	Key Concepts.....	52
5.11.5	Project 10: Program Structure Analysis	54
5.11.6	Common Student Difficulties	55
5.11.7	End-of-Lecture Summary	55
5.11.8	Preparation for Next Lecture	56
5.12	Week 13 – Lecture 11 (Chapter 11): Compiler II – Execution & Translation (Conceptual).....	56
5.12.1	Lecture Purpose	56
5.12.2	Teaching Goals (Instructor).....	56
5.12.3	Student Learning Outcomes	56
5.12.4	Key Concepts.....	57
5.12.5	Project 11: Execution Tracing & Translation Concepts	58
5.12.6	Common Student Difficulties	60
5.12.7	End-of-Lecture Summary	60
5.12.8	Preparation for Next Lecture	61
5.13	Week 14 – Lecture 12 (Chapter 12): Operating System (Conceptual)	61
5.13.1	Lecture Purpose	61
5.13.2	Teaching Goals (Instructor).....	61
5.13.3	Student Learning Outcomes	61
5.13.4	Key Concepts.....	61
5.13.5	Project 12: Operating System Services Simulation.....	65
5.13.6	Common Student Difficulties	66
5.13.7	End-of-Lecture Summary	66
5.13.8	Preparation for Final Assessment	67
6	Licensing and Usage Notes	67
7	References	0

Introduction to Computing System

(Inspired by the Nand2Tetris Course's Philosophy)

1 Course Overview

1.1 Overview

Introduction to Computing System is a foundational course designed to answer one of the most important questions in computer science:

How does a computer actually work — from logic gates to programs?

Rather than treating the computer as a black box, this course takes students on a guided journey from the simplest building block of computation, the *NAND logic gate*, to a fully functioning computer system capable of executing real programs in a simplified and conceptual form.

The course follows a *bottom-up approach* (see Figure 1). Students begin by exploring how binary values and Boolean logic form basic logic gates. These gates are then combined to construct arithmetic circuits, memory elements, and the core components of a simple CPU. Through guided construction and simulation, students learn how hardware components cooperate to perform computation.

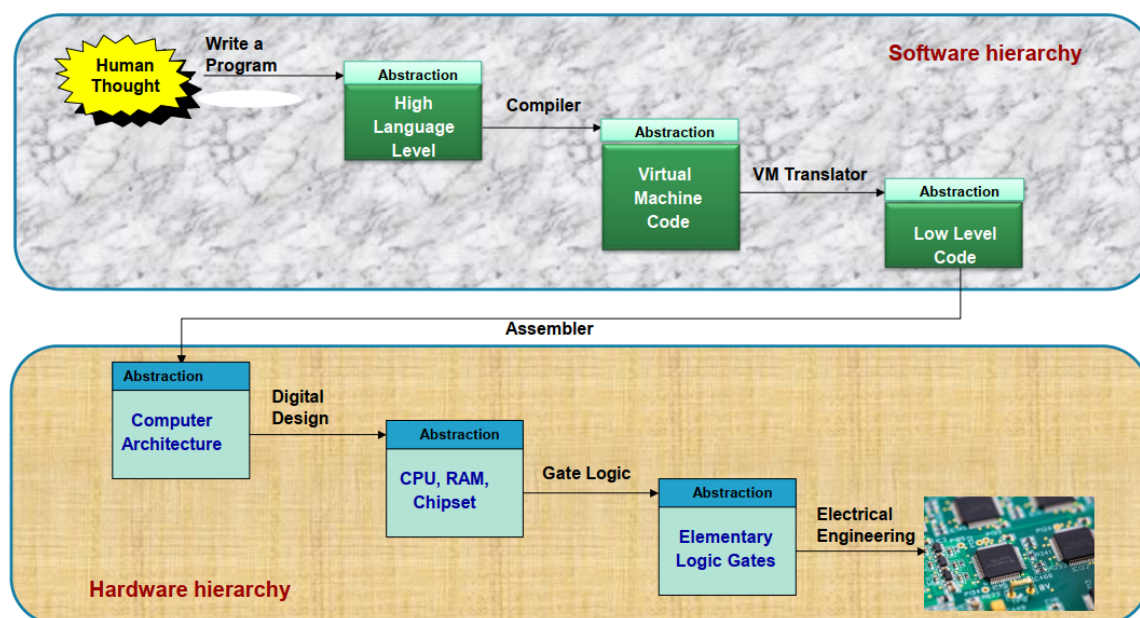


Figure 1. Hardware and software hierarchies showing how computing systems are built layer by layer, from elementary logic gates up to high-level programming abstractions.

Once the hardware foundation is established, the course shifts focus to the software stack. Students explore machine-level and assembly-level programming at a conceptual and practical level, learning how instructions control hardware execution and how control flow, functions, and data movement work at low levels. Rather than implementing full translators or compilers, students study these layers through guided projects, visual models, and step-by-step execution.

Higher-level abstractions such as virtual machines, high-level programming constructs, and operating system services are introduced to show how complex software systems are constructed from simpler execution models. Throughout the course, emphasis is placed on abstraction, system layering, and the interactions between hardware and software.

By the end of the course, students will understand how:

- Bits become logic
- Logic becomes hardware
- Hardware executes instructions
- Instructions become programs

This course is inspired by the internationally renowned *Nand2Tetris* curriculum, derived from *The Elements of Computing Systems* by Noam Nisan and Shimon Schocken.

1.2 Why This Course Is Ideal for First-Semester Freshmen

Many introductory computing courses focus on using computers, such as writing code and running programs, without explaining what happens beneath the surface. This course fills that gap.

For first-semester freshmen (including students with no prior background), this course provides:

- A clear mental model of how computers work
- Confidence that computers are engineered systems, not magic
- A strong foundation for later courses in programming, operating systems, computer architecture, and compilers

Students will learn how software executes on real hardware. They will understand why programs behave the way they do.

1.3 What This Course Will Help You Understand

By the end of the course, students will clearly understand:

1) How information is represented

- Why computers use binary values (0s and 1s)
- How numbers, instructions, and memory are encoded
- How signed numbers (negative values) and arithmetic work at the hardware level

2) **How computation happens**

- How logic gates implement Boolean functions
- How arithmetic circuits perform addition and comparison
- How the arithmetic logic unit (ALU) executes computations

3) **How memory and time work**

- How computers store and retrieve information
- What registers, RAM, and program counters are
- Why clocks are essential for coordination and synchronization

4) **How software controls hardware**

- What machine language and assembly language represent
- How instructions move data and control execution
- How loops, conditionals, and function calls are implemented at low levels

5) **How high-level programs reach hardware**

- What a virtual machine is and why it is useful
- How a compilation works conceptually, from high-level programs to executable behaviour
- How operating system services support programs execution and interaction with hardware

1.4 What You Will Build in This Course

Students will build and explore, not just study:

- Logic gates such as AND, OR, NOT, and multiplexers
- Arithmetic circuits including adders and an ALU
- Memory units such as registers, and RAM
- A complete CPU and computer architecture
- A Simple machine-level programs

Students will also explore conceptual models of:

- Assemblers
- Virtual machines
- Compilers
- Operating systems

Every major concept in the course is reinforced through hands-on, guided projects. Students focus on conceptual understanding, construction, and simulation, rather than industrial-scale or production-level implementations.

1.5 No Prior Background Required

This course is explicitly designed for students with little or no prior experience in:

-
- Programming
 - Computer hardware
 - Digital logic
 - Assembly language
 - Computer architecture

All required concepts are introduced from first principles.

1.6 Course Description

This course introduces:

- The concept of a computing system
- Hardware and software as a single integrated system
- Bottom-up computer construction from first principles
- NAND gates as a universal building block
- How this course connects to traditional CS subjects

1.7 Course Philosophy

The course is guided by:

- No black-box abstractions
- Learning by building and observing
- Clear separation between abstraction and implementation
- Modular design
- Visual reasoning before textual complexity
- Hardware–software co-design
- The *Nand2Tetris* educational philosophy, adapted for beginners

1.8 Learning Objectives

By the end of this course, students will be able to:

- Explain how data is represented and processed at the hardware level
- Design basic combinational and sequential logic circuits
- Design a complete CPU and memory system
- Write and trace simple machine-level programs
- Explain how assemblers, VMs, compilers, and OS services fit into the system stack
- Reason about program execution across abstraction layers

1.9 Prerequisites

Required

- Basic computer literacy, such as using files and folders

- Basic logical thinking using step-by-step reasoning

Helpful but not required

- Prior programming experience
- Mathematics beyond high school algebra
- Any background in electronics or hardware

Important: Students are not expected to know programming, binary math, or hardware concepts before taking this course.

1.10 Tools and Resources

- **Digital (GPLv3 Open Source) [1]** – used for logic gate construction, memory systems, and CPU datapath design (see Figure 2).
- **Little Man Computer (LMC) [2]** – used to introduce machine language execution and assembler concepts through explicit numeric instruction visualization (see Figure 3).
- **Python Tutor [3]** – used to visualize stack behavior, function calls, and step-by-step program execution at higher abstraction levels (see Figure 4).
- **Snap! [4]** – used for high-level programming, object-oriented concepts, and operating system abstractions without syntactic overhead (see Figure 5).

The original nand2tetris software suite is used conceptually, not as the primary tooling.

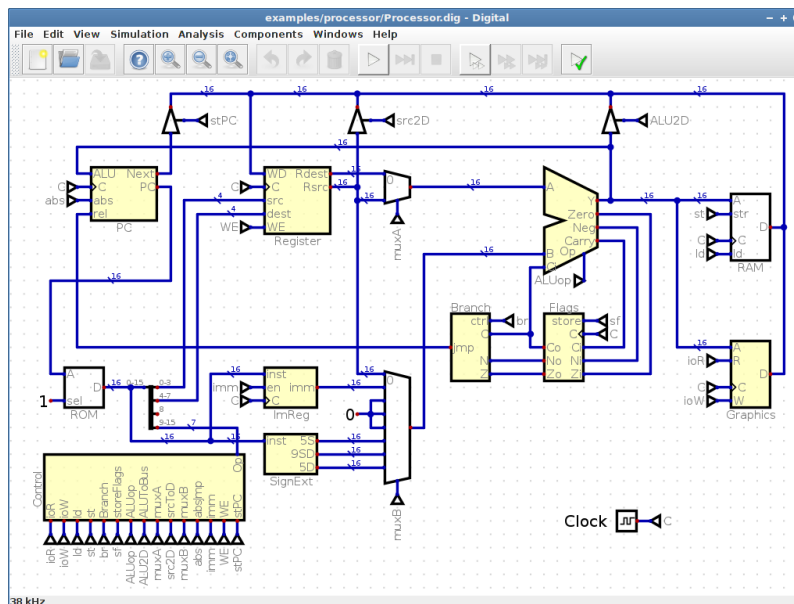


Figure 2: Screenshot of the Digital logic simulator showing a processor circuit.

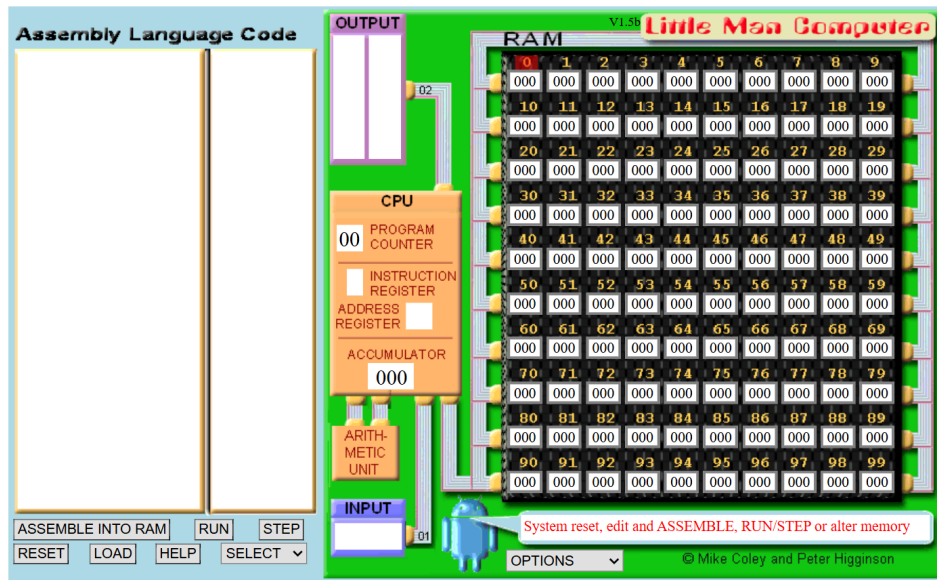


Figure 3: Screenshot of the Little Man Computer (LMC) simulator.

Online Compiler, AI Tutor, and Visual Debugger for Python, Java, C, C++, and JavaScript

Python Tutor helps you do **programming homework assignments** in Python, Java, C, C++, and JavaScript. It contains a step-by-step [visual debugger](#) and [AI tutor](#) to help you understand and debug code.

Start coding online in [Python](#), [Java](#), [C](#), [C++](#), and [JavaScript](#)

Since 2010, **over 20 million people in more than 180 countries** have used Python Tutor to visualize over 300 million pieces of code. It is the most widely-used program visualization tool for CS education.

As a preview, here is a [small example](#) that visualizes recursion in Python:

Python 3.6

```

1 def listSum(numbers):
2     if not numbers:
3         return 0
4     else:
5         (f, rest) = numbers
6         return f + listSum(rest)
7
8 myList = (1, (2, (3, None)))
9 total = listSum(myList)

```

[Edit Code & Get AI Help](#)

→ line that just executed
→ next line to execute

Step 11 of 22

Frames Objects

Global frame

- listSum
- myList
- listSum
- numbers
- f
- rest

function listSum(numbers)

- tuple (0, 1)
- tuple (1, 2)
- tuple (2, 3, None)

Figure 4: Screenshot of Python Tutor.

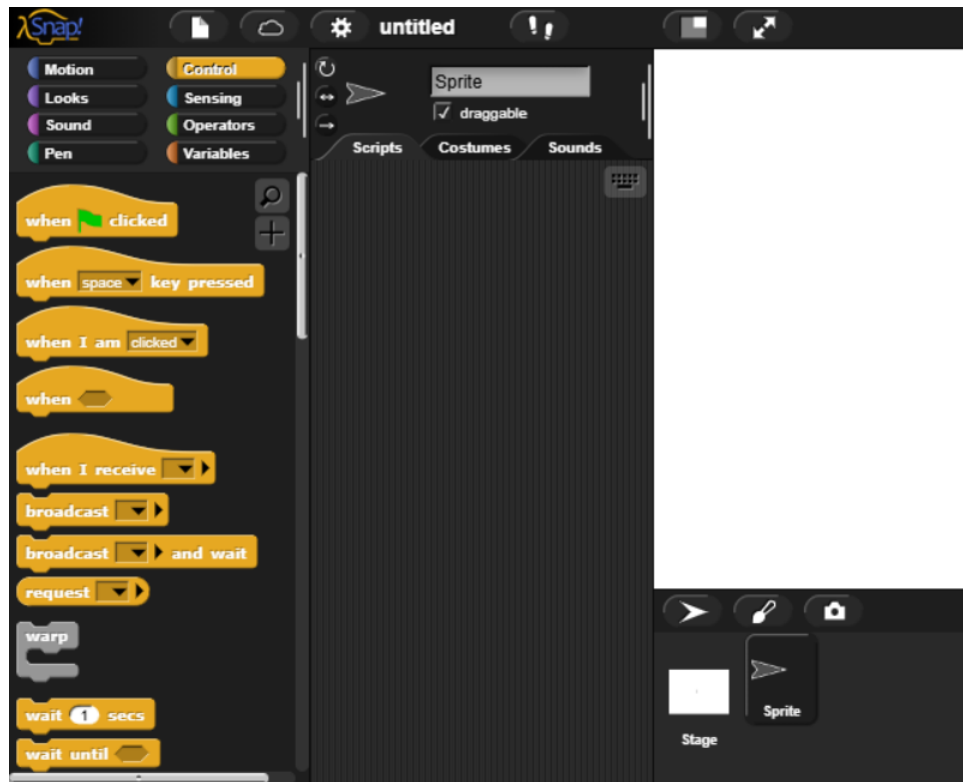


Figure 5: Screenshot of the Snap! visual programming environment.

1.11 Skills Students Will Develop

This course emphasizes thinking skills rather than memorization. Students will develop:

- Systems thinking
- Problem decomposition
- Abstraction vs. implementation awareness
- Debugging and testing discipline
- Step-by-step reasoning
- Confidence working at low levels of computing systems

These skills are valuable across many fields:

- Software engineering
- Data science
- Artificial intelligence and machine learning
- Cybersecurity
- Embedded systems

1.12 How This Course Fits into the CS Curriculum

This course serves as a foundation for:

-
- Programming languages
 - Operating systems
 - Computer architecture
 - Compilers
 - Embedded systems

Students gain an advantage because they understand why systems work, not just how to use them.

1.13 Course Structure at a Glance

- **Weeks 0–7:** Hardware and Machine Language (Logic → CPU → Assembly)
- **Week 8:** Midterm Examination
- **Weeks 9–14:** Software Stack (VM → Compiler → OS concepts)
- **Week 15:** Final Examination

Each week includes:

- Conceptual lectures
- Guided examples
- Hands-on projects
- Automated testing and feedback

1.14 What Is Expected from Students

To succeed in this course, students should:

- Attend lectures regularly
- Work consistently on weekly projects
- Be patient and persistent
- Ask questions early
- Practice incremental testing and debugging

This course is challenging, but deeply rewarding.

2 Lectures (Weekly Plan)

Table 1 outlines the week-by-week progression of the course, showing how lectures and chapters move through successive levels of abstraction from hardware foundations to high-level software concepts.

Table 1: Weekly Course Structure and Conceptual Focus

Week	Lecture	Reference Chapter	Chapter Title / Focus	Primary Purpose of the Week
------	---------	-------------------	-----------------------	-----------------------------

Week 1	Lecture 0	Introduction	Course Introduction & Computing Abstractions	Introduce course philosophy, abstraction layers, tools, and roadmap
Week 2	Lecture 1	Chapter 1	Boolean Logic	Foundations of computation using logic gates
Week 3	Lecture 2	Chapter 2	Boolean Arithmetic	Binary arithmetic and ALU design
Week 4	Lecture 3	Chapter 3	Sequential Logic	State, memory, registers, and time
Week 5	Lecture 4	Chapter 4	Machine Language (Conceptual)	Understanding machine/assembly-level execution
Week 6	Lecture 5	Chapter 5	Computer Architecture	CPU design and full computer integration
Week 7	Lecture 6	Chapter 6	Assembler (Conceptual)	Understanding assembly-to-machine translation
Week 8	—	—	Midterm Examination	Assessment of hardware & machine-level concepts
Week 9	Lecture 7	Chapter 7	Virtual Machine I: Stack Arithmetic	VM abstraction and stack-based computation
Week 10	Lecture 8	Chapter 8	Virtual Machine II: Program Control	Functions, branching, call stack, and control flow
Week 11	Lecture 9	Chapter 9	High-Level Language & OOP	High-level programming and object-oriented design
Week 12	Lecture 10	Chapter 10	Program Structure & Syntax (Conceptual)	Understanding syntax, structure, and program organization
Week 13	Lecture 11	Chapter 11	Execution & Translation (Conceptual)	How high-level programs execute step by step
Week 14	Lecture 12	Chapter 12	Operating System (Conceptual)	OS services, abstraction, and runtime support
Week 15	—	—	Final Examination	---

3 Projects

Table 2 summarizes the sequence of projects in the course, showing how each project aligns with its corresponding lecture, conceptual focus, and supporting tool across the semester.

The project sequence is designed according to the *nand2tetris* philosophy, progressing systematically from low-level hardware construction to high-level software abstraction. Early projects use Digital to build computation bottom-up from logic gates through memory and a complete CPU, making hardware structure, datapaths, and control logic explicit. Little Man Computer (LMC) is used for machine language and assembler projects, where the emphasis is on instruction execution, numeric machine code, and symbol resolution rather than hardware implementation. The course then transitions to higher abstraction layers: Python Tutor is used to visualize virtual machines, stack behavior, and execution flow, while Snap! supports high-level programming, object-oriented design, and operating system concepts without syntactic overhead. Tool transitions are intentional and reflect shifts in abstraction level, ensuring that each project foregrounds its core learning objective while maintaining conceptual continuity across the hardware–software stack.

Table 2: Project Sequence, Learning Focus, and Tool Alignment

Week	Lecture	Chapter	Project #	Project Title	What Students Build / Do (Deliverables)	Core Concepts & Learning Focus	Tool(s)
Week 1	Lecture 0	—	—	—	Course setup, abstraction overview	Hardware–software layers	—
Week 2	Lecture 1	Ch. 1	Project 1	Boolean Logic	Logic gates from NAND (NOT, AND, OR, XOR, MUX/DMUX)	Boolean logic, NAND universality, abstraction	Digital
Week 3	Lecture 2	Ch. 2	Project 2	Boolean Arithmetic	Adders, incrementer, comparisons, ALU	Binary arithmetic, ALU design	Digital
Week 4	Lecture 3	Ch. 3	Project 3	Memory	Latches, flip-flops, registers, RAM, counter, PC	State, clocking, memory hierarchy	Digital
Week 5	Lecture 4	Ch. 4	Project 4	Machine Language (Conceptual)	Write & step through simple assembly programs	Instruction execution, control flow	Little Man Computer
Week 6	Lecture 5	Ch. 5	Project 5	Computer Architecture	Integrated CPU + memory datapath	Datapath, control logic, system integration	Digital
Week 7	Lecture 6	Ch. 6	Project 6	Assembler	Assemble symbolic programs → numeric machine code (two-pass concept)	Translation, symbols, machine code	Little Man Computer (LMC)
Week 8	—	—	—	Midterm Exam	Covers Projects 1–6	Hardware + machine-level understanding	—
Week 9	Lecture 7	Ch. 7	Project 7	VM I: Stack Arithmetic	Simulate push/pop & arithmetic	Stack machines, VM abstraction	Python Tutor
Week 10	Lecture 8	Ch. 8	Project 8	VM II: Program Control	Simulate calls, returns, stack frames	Call stack, function frames	Python Tutor
Week 11	Lecture 9	Ch. 9	Project 9	High-Level Language & OOP	Object-based program (game / simulation)	Encapsulation, inheritance, polymorphism, abstraction	Snap!
Week 12	Lecture 10	Ch. 10	Project 10	Program Structure & Syntax	Structured block programs	Syntax, program structure	Snap!
Week 13	Lecture 11	Ch. 11	Project 11	Execution & Translation Concepts	Step-by-step execution visualization	How high-level code executes	Python Tutor
Week 14	Lecture 12	Ch. 12	Project 12	Operating System (Conceptual)	Mini OS with services (system calls)	OS services, abstraction	Snap!
Week 15	—	—	—	Final Exam	Comprehensive exam	Full hardware–software stack	—

The project sequence follows a bottom-up hardware-to-software progression consistent with *Nand2Tetris*, with tools selected to match the abstraction level of each stage. *Digital* is used in Projects 1–3 and Project 5 to build computation from logic gates up through memory and a complete CPU, making hardware structure, datapaths, and control logic explicit. *Little Man Computer (LMC)* is used in Projects 4 and 6 when the focus shifts to machine language execution and assembly translation, because it clearly exposes instruction sequencing, numeric machine code, and symbol resolution without hardware complexity. The course then moves to higher abstraction

levels: *Python Tutor* is used to visualize virtual machines, stack behavior, and execution flow, while *Snap!* supports high-level programming, object-oriented concepts, and operating system ideas without syntactic overhead. Switching tools is intentional and reflects a change in perspective—hardware construction, program execution, translation, and abstraction—ensuring that each project emphasizes the core concept of its chapter with minimal cognitive load.

4 Assessment & Grading Breakdown

Table 3 presents the assessment components for the course and their corresponding weightings, outlining how student performance is evaluated across participation, examinations, and project-based work.

Table 3: Assessment and Grading Breakdown

Assessment Component	Description	Weight (%)
Attendance & Participation	Attendance and active participation during lectures and project/lab sessions	20
Midterm Examination	Covers hardware and machine-level concepts (Chapters 1–6)	20
Projects	Weekly hands-on projects (hardware, VM, compiler, operating system)	20
Final Examination	Comprehensive exam covering the full hardware–software stack	20
Quizzes / Presentations / Other Assessments	Short quizzes, presentations, or supplementary evaluations	20
Total		100

4.1 Attendance & Participation Policy (20%)

- Each class session is **3 hours**
(1 hour lecture + 2 hours project/lab)
- Each session is divided into **6 equal segments**
(30 minutes per segment)
- Each segment is worth **0.65 points**
- **Total attendance points per class: 3.9 points**

Late Policy

- Missing any **30-minute** segment results in a loss of **0.65 points**
- Points are deducted proportionally based on arrival and departure time

Absence

- Missing the entire 3-hour session results in a loss of **all 3.9 points for that day**

Attendance is recorded throughout the entire session, including both lecture and project/lab time.

4.1.1 General Notes

- Projects are cumulative and build on previous weeks
- Regular attendance is essential due to the incremental nature of the course
- Exams emphasize **conceptual understanding**, not memorization
- Quizzes and presentations reinforce weekly concepts and communication skills

5 Weekly Course Breakdown

5.1 Week 1 – Lecture 0: Course Introduction & Computing Abstractions

5.1.1 Lecture Purpose

Introduce the scope, philosophy, and structure of the course. Establish a systems mindset, explain the bottom-up approach, and familiarize students with modern, visual tools used throughout the course. No technical mastery is expected.

5.1.2 Teaching Goals (Instructor)

- Present the big-picture roadmap of the course
- Frame hardware and software as one integrated system
- Introduce abstraction layers in computing systems
- Explain the bottom-up methodology
- Given a conceptual overview of a simple computer system
- Reduced anxiety by introducing visual, beginner-friendly tools

5.1.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Describe what a computer is (input, processing, memory, output)
- Explain abstraction layers in a computing system
- Understand the bottom-up approach used in the course
- Identify major components of a simple computer (conceptually)
- Open and explore the course tools at a basic level

5.1.4 Key Concepts

1. What Is a Computer?

A computer is a system that accepts input, processes data, stores information, and produces output. Complex behavior emerges from simple, rule-based components.

2. Hardware and Software as One System

Software controls hardware; hardware constrains software. Neither is meaningful in isolation.

3. Abstraction Layers

Computing systems are built in layers ([high-level languages](#) → VM → [assembly](#) → [machine language](#) → [hardware](#)). Each layer hides complexity while relying on the one below.

4. Bottom-Up Approach

The course builds each layer from scratch, starting with simple components and progressively constructing a complete system.

5. Why NAND Gates?

NAND is a universal logic gate. Any computation can be built from it, making it an ideal foundational primitive.

6. The Computer We Will Build (Overview)

A simple but complete computer consisting of a CPU, memory, instruction storage, program counter (PC), control logic, and basic I/O. Students will build this system during the course.

7. Course Roadmap

- Weeks 1–7: Hardware and machine language
- Week 8: Midterm
- Weeks 9–14: Software stack
- Week 15: Final

5.1.5 Tools Introduction

- **Digital** – build hardware visually
- **Nandgame** – intuitive logic warm-up
- **Little Man Computer** – machine language & assembler concepts
- **Python Tutor** – stack, calls, execution
- **Snap!** – high-level programming, OOP, OS concepts

Goal: build confidence and reduce tool-related anxiety.

5.1.6 Activities

- Open [Digital](#) and explore a simple logic gate
- Try [Nandgame](#) first levels (optional)
- Run a simple [LMC](#) program and step through execution
- Open [Python Tutor](#) and step through a short program
- Explore [Snap!](#) interface (no coding yet)

5.1.7 Optional **Project 0** (Highly Recommended)

Purpose: familiarize students with tools and workflow.

Focus: Opening tools, running examples, observing behaviour, no design or coding complexity.

5.1.8 Common Misconceptions (Address Explicitly)

- Prior programming experience is required → False
- Hardware is inherently too difficult → False
- Understanding everything immediately is expected → False

5.1.9 End-of-Lecture Summary

Students should leave with a clear understanding of:

- What the course is about
- How it is structured
- What they will build
- How to use the tools at a basic level

5.1.10 Preparation for Next Lecture

- Read Chapter 1 (Boolean Logic)
- Confirm tools are installed and working
- Review abstraction layers discussed in class

5.2 Week 2 – Lecture 1 (Chapter 1): Boolean Logic

5.2.1 Lecture Purpose

Introduce the logical foundations of computation. Students learn how **simple logic gates** can be combined to perform meaningful computation and how **NAND alone** is sufficient to build all other logic components.

5.2.2 Teaching Goals (Instructor)

- Establish Boolean logic as the basis of digital computation
- Introduce logic gates as physical computing elements
- Emphasize abstraction: *what a chip (circuit) does vs. how it is built*
- Prepare students for hardware description language (HDL)
- Connect logic theory to hands-on circuit construction

5.2.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

-
- Explain Boolean values and truth tables
 - Describe basic Boolean operations (AND, OR, NOT)
 - Understand NAND as a universal gate
 - Understand the purpose of multiplexers and demultiplexers
 - Build and test basic logic circuits visually

5.2.4 Key Concepts

1. Boolean Values and Logic

- Computers represent information using binary values: 0 and 1
- Boolean logic defines operations on binary values
- Truth tables describe how logic functions behave

2. Boolean Operations

- AND, OR, NOT as fundamental logical operations
- Logical equivalence between expressions
- Relationship between logic and decision-making

3. Logic Gates

- Physical implementations of Boolean operations
- Gates operate on bits and output bits
- Gates are building blocks for all digital systems

4. NAND as a Universal Gate

- NAND can implement NOT, AND, OR, XOR
- Any Boolean function can be built using only NAND
- Using a single primitive simplifies hardware design

5. Multi-Bit Gates and Buses

- Gates can operate on multi-bit inputs
- Buses group multiple wires into a single abstraction
- Parallel bit-wise computation

6. Control Gates

- Multiplexers (Mux): select one input from many
- Demultiplexers (DMux): route input to one of many outputs
- Foundation for control logic and data routing

7. Abstraction vs. Implementation

- Chips are defined by behavior, not internal structure
- A chip can be built from simpler chips
- Enables hierarchical system design

5.2.5 Project 1: Elementary Logic Gates

Goal

Build a complete set of basic logic gates visually, starting from NAND..

Chips Implemented

- Not,
- And
- Or
- Xor
- Mux
- DMux
- Not16
- And16
- Or16
- Mux16
- Or8Way
- Mux4Way16
- Mux8Way16
- DMux4Way
- DMux8Way
- (Nand is provided as a primitive and is not implemented.)

5.2.6 Project Focus

- Correct functional behavior
- Understanding what the circuit does, not just wiring
- Respecting abstraction boundaries
- Incremental testing and validation

5.2.7 Tools Used

Students may choose one of the following:

Digital

- Schematic-based, formal circuit design
- Explicit wiring of gates and buses
- Prepare students for later hardware construction projects

Optional Alternative: Nandgame [5] may be used as an introductory visualization tool to facilitate conceptual understanding of NAND-based logic before students transition to formal schematic-based circuit design in Digital (see Figure 6).

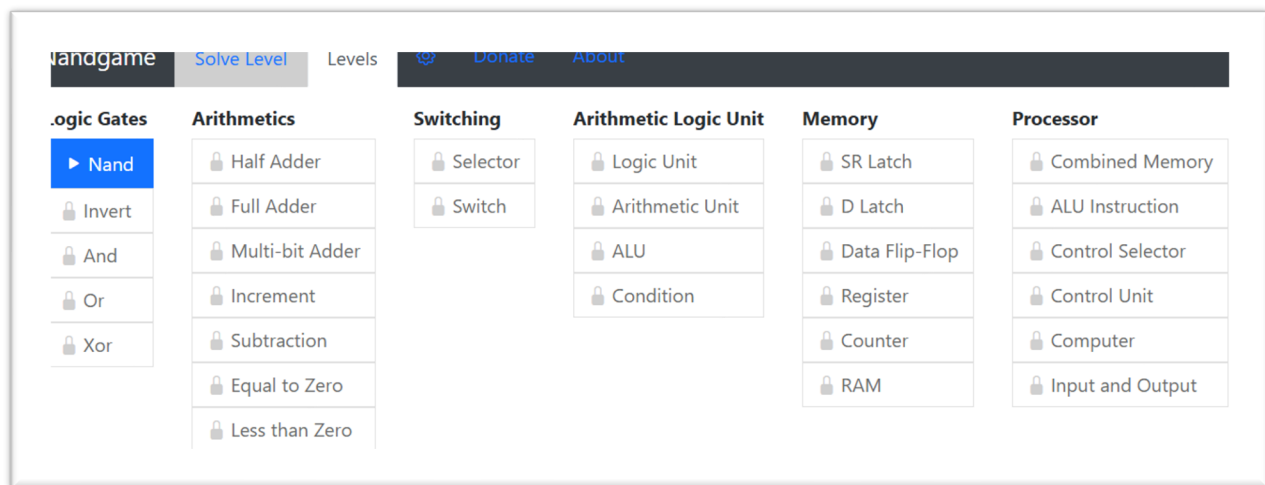


Figure 6: Screenshot of the Nandgame logic gate simulation interface.

5.2.8 In-Class Activities

- Build NOT, AND, OR using only NAND
- Toggle inputs and observe outputs
- Verify behavior using truth tables
- Compare single-bit and multi-bit gates

5.2.9 Common Student Difficulties

- Confusing truth tables with implementation
- Thinking NAND is “special” rather than universal
- Forgetting abstraction boundaries
- Debugging without testing incrementally

Instructor tip: Emphasize “*Build small, test early, test often.*”

5.2.10 End-of-Lecture Summary

Students now understand how **simple logic gates** form the foundation of all computation and how complex behavior emerges from simple Boolean operations.

5.2.11 Preparation for Next Lecture

- Review binary number representation
- Read Chapter 2 (Boolean Arithmetic)
- Complete Project 1 incrementally

5.2.12 Resources

- **Core Reading**
Elements of Computing Systems — Chapter 1: Boolean Logic

https://b1391bd6-da3d-477d-8c01-38cdf774495a.filesusr.com/ugd/44046b_f2c9e41f0b204a34ab78be0ac4953128.pdf

- **Supplementary Reading**
All About Circuits — Digital Logic (logic gates, truth tables, combinational logic)
<https://www.allaboutcircuits.com/textbook/digital/>
- **Introductory Video**
Crash Course Computer Science #3 — Logic Gates
<https://www.youtube.com/watch?v=QZwneRb-zqA>
- **Interactive Practice**
CircuitVerse — Logic gates and combinational logic
<https://learn.circuitverse.org/>
- **Tool Tutorial (Optional)**
Hneemann Digital — Logic gates, analysis, and synthesis walkthrough
<https://www.youtube.com/watch?v=agO4eO-bamk>
- **Simulation Tool**
Hneemann Digital — Download and example circuits
<https://github.com/hneemann/Digital>

5.3 Week 3 – Lecture 2 (Chapter 2): Boolean Arithmetic

5.3.1 Lecture Purpose

Introduce how computers perform arithmetic operations using binary logic. This lecture connects logic gates to numeric computation and shows how arithmetic circuits are constructed from simple components.

5.3.2 Teaching Goals (Instructor)

- Explain binary number representation
- Introduce signed numbers using two's complement
- Show how addition is implemented in hardware
- Present the ALU as the core computational unit of a CPU
- Reinforce hierarchical design and abstraction

5.3.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Represent numbers in binary
- Explain two's complement representation
- Describe how half adders and full adders work
- Explain how multi-bit addition is built from simple adders
- Understand the role of the ALU and its control signals

5.3.4 Key Concepts

1. Binary Number Representation

- Positional representation using base 2
- Each bit represents a power of two
- n-bit numbers have a fixed numeric range
- Binary addition follows simple, local rules

2. Signed Numbers and Two's Complement

- Why computers need to represent negative numbers
- Two's complement:
 - Uses the same hardware for addition & subtraction
 - Has only one representation of zero
- Overflow exists and must be detected, not avoided

Focus on intuition, not proofs.

3. Half Adders and Full Adders

- Half adder: sum of two bits
- Full adder: sum with carry-in
- Building larger adders from smaller ones

4. Multi-Bit Addition

- Ripple-carry adders
- Add16 and incrementor circuits
- Structural implications of multi-bit addition (conceptual)

5. Arithmetic Logic Unit (ALU)

- Performs arithmetic and logical operations
- Controlled by operation bits
- Produces result and status flags

6. Abstraction and Reuse

- Small chips composed into larger systems
- Clear interfaces simplify design and debugging

5.3.5 Project 2: Arithmetic Logic Unit

Goal

Build arithmetic components that culminate in a functional Arithmetic Logic Unit, using visual circuit design.

Chips Implemented

- HalfAdder, FullAdder
- Multi-bit Adder (e.g., 8-bit or 16-bit)
- Incrementer

-
- ALU

Project Focus

- Correct arithmetic behavior
- Correct carry propagation
- Understanding ALU control signals
- Incremental construction and testing

Tools Used

Students may choose one of the following:

Digital

- Schematic-based, formal circuit design
- Better preparation for later hardware projects

Optional Alternative: Nandgame may be used for initial visualization of basic arithmetic circuits before transitioning to full schematic-based ALU construction in Digital.

5.3.6 In-Class Activities

- Build a half adder live
- Trace carry propagation by hand
- Toggle ALU control bits and observe outputs
- Compare addition vs. subtraction visually

5.3.7 Common Student Difficulties

- Confusing unsigned and signed numbers
- Misunderstanding overflow behaviour
- Incorrect carry propagation
- Building large circuits without testing subcomponents

Instructor tip: *Always test the smallest circuit before composing it into a larger one.*

End-of-Lecture Summary

Students now understand how computers perform arithmetic operations using logic gates and how the ALU serves as the core computational engine of the CPU.

5.3.8 Preparation for Next Lecture

- Review clocked vs. unclocked systems
- Read Chapter 3 (Sequential Logic)
- Complete Project 2 incrementally

5.3.9 Resources

-
- **Core Reading**
Elements of Computing Systems — Chapter 2: Boolean Arithmetic
https://www.cs.huji.ac.il/course/2002/nand2tet/oldsite/docs/ch_2_boolean_arithmetic.pdf
 - **Supplementary Reading**
All About Circuits — Digital Number Systems & Binary Arithmetic
<https://www.allaboutcircuits.com/textbook/digital/>
 - **Introductory Video (Overview)**
RF Design Basics — Binary Addition, Subtraction, Multiplication, Division
<https://www.youtube.com/watch?v=-f6fjBhu8eA>
 - **Conceptual Video (Signed Numbers & Two's Complement)**
The Organic Chemistry Tutor — Binary Addition & Subtraction with Negative Numbers
<https://www.youtube.com/watch?v=sJXTo3EZoxM>
 - **Conceptual Video (Two's Complement Focus)**
Two's Complement & Signed Arithmetic Explanation
<https://www.youtube.com/watch?v=AHQwhdz3tlg>
 - **Interactive Practice (Optional)**
CircuitVerse — Binary adders and arithmetic circuits
<https://learn.circuitverse.org/>
-

5.4 Week 4 – Lecture 3 (Chapter 3): Sequential Logic: Registers and Memory

5.4.1 Lecture Purpose

Introduce state and memory in computing systems. This lecture explains how computers remember information over time and how clocked components enable sequential behavior, as opposed to purely combinational logic.

5.4.2 Teaching Goals (Instructor)

- Distinguish **combinational logic** from **sequential logic**
- Introduce the concept of **time** and **state** in computation
- Explain the role of clocks in digital systems
- Show how memory hierarchies are built from simple components
- Prepare students to design registers, RAM, and program counters

5.4.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain the difference between combinational and sequential circuits
- Describe how clocks control state changes
- Understand how bits are stored and updated
- Explain how registers and RAM work conceptually
- Understand the purpose of the program counter (PC)

5.4.4 Key Concepts

1. Combinational vs. Sequential Logic

- Combinational circuits depend only on current inputs
- Sequential circuits depend on inputs and past state
- Memory is required to store state

2. The Clock

- A clock coordinates when state changes occur
- State updates happen only on clock ticks
- Prevents uncontrolled feedback and race conditions

3. Storing a Single Bit

- A bit stores one binary value (0 or 1)
- Implemented using a clocked storage element (DFF)
- Foundation of all memory elements

4. Registers

- A register stores multiple bits as a single word
- Supports load and hold operations
- Used for temporary storage inside the CPU

5. Random Access Memory (RAM)

- Memory organized as an array of registers
- Address selects which register to access
- Hierarchical construction:
 - RAM8 → RAM64 → RAM512 → RAM4K → RAM16K

6. Program Counter (PC)

- Holds the address of the next instruction
- Supports increment, load, and reset
- Central to instruction sequencing

7. Abstraction and Hierarchical Design

- Larger memory units built from smaller ones
- Clear interfaces simplify reasoning and testing

5.4.5 Project 3: Memory

Goal

Build a complete memory subsystem using visual, clocked circuit design

Chips Implemented

- Bit (single-bit storage)
- Register

-
- Small RAM (e.g., RAM8, RAM64)
 - Larger RAM (conceptual extension)
 - Program Counter (PC)

Project Focus

- Correct clocked behavior
- Proper load vs. hold control
- Correct address selection
- Incremental testing of memory components

Tools Used

Digital

Why Digital:

- Explicit clock visualization
- Clear separation of combinational vs. sequential logic
- Excellent for memory and PC design
- No syntax overhead

5.4.6 In-Class Activities

- Demonstrate a register holding value across clock ticks
- Toggle clock manually and observe state change
- Compare behavior with and without clock
- Step through PC increment and load operations

5.4.7 Common Student Difficulties

- Forgetting that state changes only on clock ticks
- Confusing load vs. hold behavior
- Expecting memory to behave like combinational logic
- Building large RAM blocks without testing smaller ones

Instructor tip: repeatedly emphasize *“state changes only on the clock.”*

5.4.8 End-of-Lecture Summary

Students now understand how computers store information, manage time, and maintain state, enabling programs to execute step by step rather than all at once.

5.4.9 Preparation for Next Lecture

- Review how instructions interact with memory
 - Read Chapter 4 (Machine Language)
 - Begin Project 3 early and test incrementally
-

5.5 Week 5 – Lecture 4 (Chapter 4): Machine Language

5.5.1 Lecture Purpose

Introduce machine language and assembly language programming and show how software controls hardware at the lowest level. Students learn how instructions are represented, executed, and sequenced on a simple computer model, connecting previously built hardware concepts to executable programs.

5.5.2 Teaching Goals (Instructor)

- Explain what machine language is and why it exists
- Explain the role of assembly language as a readable form of machine language
- Introduce a simple computer model suitable for beginners
- Demonstrate how instructions control:
 - Computation
 - Memory
 - Control flow
- Prepare students to understand CPU behavior before building one in Project 5

5.5.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain what machine language is and how it differs from assembly language
- Describe the components of a simple computer (memory, accumulator, PC)
- Explain how instructions are fetched and executed step by step
- Understand how control flow (loops and decisions) is implemented
- Write and trace simple assembly programs

5.5.4 Key Concepts

1. What Is Machine Language?

- Machine language is the only language hardware executes directly
- Instructions are encoded as numeric values
- Each instruction causes a small, precise change in machine state

Machine language is about **control**, not convenience.

2. A Simple Computer Model

- **Registers:** A-register, D-register
- **Memory:** RAM (stores data), ROM (stores program instructions)
- **Program Counter (PC):** tracks the address of the next instruction
- Programs execute one instruction at a time in a fetch-and-execute sequence

The Little Man Computer has a single general-purpose register called the accumulator (ACC).

3. Assembly Language

- Assembly language uses **mnemonics** instead of numbers
- Example:

```
LDA 05
ADD 06
OUT
```
- Each mnemonic maps directly to a machine instruction

An assembler translates assembly into numeric machine code.

4. Memory and Data Access

- Instructions operate on:
 - The accumulator
 - Memory locations
- Memory stores:
 - Program instructions
 - Program data

Programs are just data interpreted as instructions

5. Control Flow

- Programs normally execute sequentially
- Control flow changes using:
 - Branch instructions
 - Conditional execution
- Enables:
 - Loops
 - If/else behavior

Labels provide symbolic names for jump targets.

6. Assembly Language Abstraction

- Assembly language uses symbolic names instead of numeric addresses
- Symbols improve readability and programmability
- An assembler translates symbolic assembly instructions into machine code
- In this chapter, students write assembly programs without building the assembler

5.5.5 Project 4: Machine Language Programming

5.5.6 Project Goal

Write and execute simple assembly programs on a minimal computer model and trace instruction execution step by step.

5.5.7 What Students Do

Using **Little Man Computer**, students:

- Write assembly programs
- Assemble them into **numeric machine code**
- Step through execution
- Observe:
 - ACC
 - PC
 - Memory changes

Alternative Tool Option: Projects in this chapter may also be completed using a simple *RISC simulator* [6] (see Figure 7). In this case, the accumulator (ACC) corresponds to a single general-purpose register (e.g., R0). This option is provided to allow instructors to align early machine-language concepts with later register-based CPU models, without changing the learning objectives of the project.

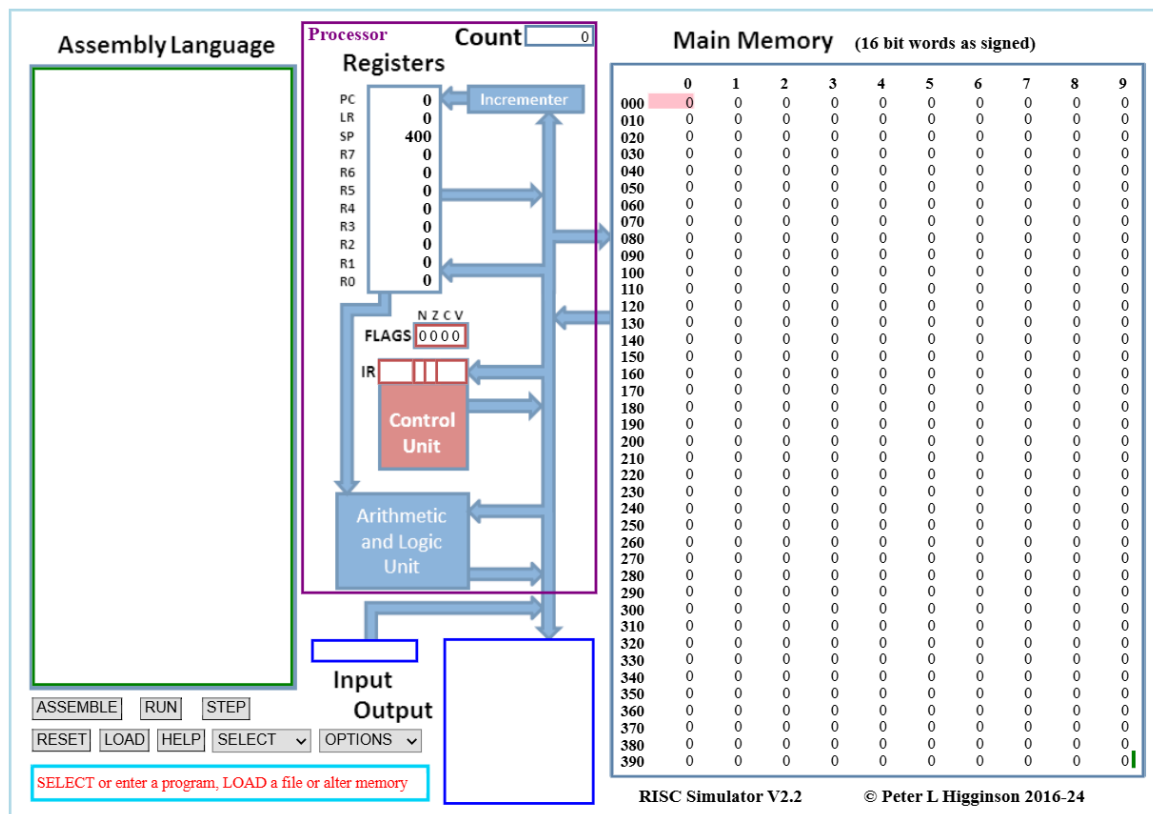


Figure 7: Screenshot of the RISC Simulator.

5.5.8 Required Programs

i. Add Program

- Add two constants (e.g., 5 + 6)
- Output the result

Example:

```
LDA FIVE
ADD SIX
OUT
HLT

FIVE DAT 5
SIX  DAT 6
```

ii. Loop Program

- Count down from a number
- Demonstrate branching and looping

iii. Conditional Program

- If input is zero → output 0
- Else → output 1

(Program size and complexity are intentionally small.)

Project Focus

- Correct sequencing of instructions
- Correct use of memory and accumulator
- Understanding control flow
- Step-by-step tracing of execution

5.5.9 Tools Used

- **Little Man Computer**

Why LMC:

- Shows machine code explicitly
- Visual execution
- Minimal syntax
- Perfect for first exposure to machine language
- Common Student Difficulties (Address Explicitly)
- Expecting multiple things to happen at once
- Losing track of accumulator state
- Confusing instructions with data
- Skipping execution tracing

5.5.10 Common Student Difficulties

- Expecting multiple things to happen at once
- Losing track of accumulator state
- Confusing instructions with data
- Skipping execution tracing

Instructor tip: Push students to say out loud: *“What changed after this instruction?”*

5.5.11 End-of-Project Summary

Students now understand:

- How programs control hardware
- What machine language really is
- How instructions execute step by step
- Why CPUs need registers, memory, and a PC

5.5.12 Preparation for Next Lecture

- Complete and thoroughly test Project 4
- Review how instructions are fetched and executed conceptually
- Read Chapter 5, *Computer Architecture*

5.6 Week 6 – Lecture 5 (Chapter 5): Computer Architecture

5.6.1 Lecture Purpose

Explain how a complete computer system is constructed by integrating the CPU, memory, and control logic. This lecture shows how machine instructions are fetched, interpreted, and executed at the hardware level, bringing together all components built so far.

5.6.2 Teaching Goals (Instructor)

- Explain how a CPU is built from the ALU, registers, and control logic
- Explain the hardware-level instruction execution cycle
- Show how datapaths and control signals coordinate computation
- Integrate memory, CPU, and instruction storage into a full system
- Prepare students to build and test a working computer

5.6.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Identify the major components of a simple CPU
- Explain how a machine instruction is executed in hardware
- Describe how ALU, registers, memory, and the Program Counter interact
- Explain the role of control logic in coordinating the system
- Trace instruction execution through a complete computer

5.6.4 Key Concepts

1. CPU Components (CPU)

-
- Arithmetic Logic Unit (ALU)
 - Registers, including A-register and D-register
 - Program Counter (PC)
 - Control logic derived from instruction bits

Note: Instruction decoding is not a separate unit, but is implemented implicitly through wiring and control signals.

2. Instruction Execution Cycle

- Fetch instruction from instruction memory (ROM32K)
- Decode instruction fields to generate control signals
- Execute computation or memory access using the ALU
- Update registers or data memory as specified
- Update the Program Counter based on jump conditions

3. Data Path

- Flow of data between registers, ALU, and memory
- Controlled by instruction bits and control logic
- Determines where values are computed, stored, or forwarded

4. Control Logic

- Generates control signals from instruction fields
- Controls ALU operation, register loading, memory writes, and jumps
- Coordinates combinational and sequential components across clock cycles

5. Memory Integration

A complete computer includes:

- Instruction memory (read-only, sequential fetch)
- Data memory (read/write)
- Address selection provided by:
 - PC (instructions)
 - Address registers (data)

Memory and CPU operate together every clock cycle.

6. System Integration

- CPU, memory, and instruction storage are integrated into **one system**
- All components from previous projects come together
- The final system can:
 - Fetch instructions
 - Execute them
 - Update state

-
- Repeat continuously

This is the first complete computer students build.

5.6.5 Project 5: Computer

Project Goal

Build a complete working computer capable of executing machine language programs by implementing three chips: Memory, CPU, and instruction memory.

Chips Built

- ALU (from Project 2)
- **Registers & Program Counter** (from Project 3)
- CPU (datapath + control logic)
- **Memory subsystem**
- **Computer** (full integration)

Project Focus

- Correct hardware-level instruction execution
- Proper generation and propagation of control signals
- Correct Program Counter update behavior
- Incremental integration and testing

Tools Used

- Digital

Why Digital:

- Supports gate-level and component-level hardware construction
- Enables explicit datapath and control logic design
- Visualizes clocked behavior and signal propagation
- Matches the hardware-first goals of this chapter

5.6.6 In-Class Activities

- Walk through one instruction cycle visually
- Highlight which wires are active
- Toggle control signals and observe effects
- Trace PC behavior for sequential and jump instructions

5.6.7 Common Student Difficulties

- Incorrect wiring between CPU subcomponents
- Errors in control signal logic
- Program Counter update mistakes

-
- Attempting full integration before testing subcomponents

Instructor tip: *Test every component independently before integration*

5.6.8 End-of-Lecture Summary

Students now understand how individual hardware components are integrated to form a complete computer system capable of executing machine language programs.

5.6.9 Preparation for Next Lecture

- Complete and thoroughly test Project 5
 - Review how machine language instructions control hardware
 - Read Chapter 6, *Assembler*
 - Prepare to transition from hardware construction to software translation
-
-

5.7 Week 7 – Lecture 6 (Chapter 6): Assembler

5.7.1 Lecture Purpose

Introduce the assembler as a translation layer between human-readable assembly programs and machine-executable instructions. Students learn why assemblers exist, how symbols are resolved, and how assembly becomes machine code, using a simple, visual computer model.

5.7.2 Teaching Goals (Instructor)

- Explain why assemblers are necessary
- Clarify the gap between symbols and numeric machine code
- Introduce labels and variables
- Explain the two-pass assembler idea
- Show how assembly instructions become machine instructions
- Reinforce the software–hardware interface

5.7.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain what an assembler does and why it is needed
- Distinguish assembly language from machine language
- Explain how symbols and labels are resolved
- Describe the two-pass assembler strategy
- Understand how assembly is translated into numeric machine code

(Students do not implement an assembler in code.)

5.7.4 Key Concepts

1. Why an Assembler Is Needed

- Machine language is binary and difficult for humans to write and read
- Assembly language introduces symbols and mnemonics
- The assembler translates human-readable assembly into machine-executable binary code

2. Assembly vs. Machine Language

- **Assembly language:** symbolic, readable
- **Machine language:** numeric, executable

Example (LMC):

Assembly:

```
LDA COUNT
ADD ONE
OUT
```

Machine code (after assembly):

```
504
106
902
```

3. Symbols and Labels

- **Labels** name instruction addresses
- **Variables** name memory locations
- Symbols improve:
 - Readability
 - Maintainability

The assembler replaces symbols with **numbers**

4. Two-Pass Assembler Design

- **First pass:** scan the program and record label definitions without generating code
- **Second pass:** translate instructions into binary while resolving symbols
- This design handles forward references correctly

5. Instruction Translation

For each assembly instruction:

- Identify the operation
- Identify any memory address
- Encode as a numeric instruction

In LMC:

-
- First digit = opcode
 - Last two digits = address

6. Software–Hardware Interface

- Assembler output = machine code
- Machine code is:
 - Stored in memory
 - Executed by hardware
- This completes the path:
- Human-readable → Machine-executable

5.7.5 Project 6: Assembler

Project Goal

Understand and demonstrate how symbolic assembly programs are translated into machine code using a simple computer model

What Students Do

Using [Little Man Computer](#), students:

- Write assembly programs with:
 - Labels
 - Variables
- Assemble them into numeric machine code
- Inspect memory after assembly
- Explain how symbols were resolved

Required Tasks

i. Task 1: Symbolic Program

- Write an assembly program using:
 - At least one label
 - At least one variable

ii. Task 2: Assembly → Machine Code

- Assemble the program
- Record:
 - Symbol table (label → address)
 - Final numeric instructions

iii. Task 3: Explanation

-
- Short written explanation:
 - What happened in pass 1
 - What happened in pass 2

What Is NOT Required

- Writing an assembler in Python/Java/etc.
 - Producing binary bit patterns
 - Error handling
-

Tools Used

- **Little Man Computer**

Why LMC:

- Explicit machine code
- Visual symbol resolution
- No syntax overload
- Perfect conceptual assembler model

5.7.6 Common Student Difficulties

- Confusing labels with variables
- Thinking translation happens in one step
- Forgetting that symbols do not exist in hardware
- Treating machine code as “magic numbers”

Instructor tip: *Make students write the symbol table by hand once*

5.7.7 End-of-Lecture Summary

Students now understand:

- Why assemblers exist
- How symbols are resolved
- How assembly becomes machine code
- How software connects to hardware

This completes the hardware → software bridge.

5.7.8 Preparation for Next Lecture

- Complete and thoroughly test Project 6
- Review how machine language programs run on the Hack platform

-
- Prepare for the midterm examination (if scheduled here)
 - Read Chapter 7, *Virtual Machine I*
-

5.8 Week 9 – Lecture 7 (Chapter 7): Virtual Machine I: Stack Arithmetic

5.8.1 Lecture Purpose

Introduce the Virtual Machine (VM) abstraction as a bridge between high-level languages and low-level execution. This lecture focuses on stack-based computation, showing how arithmetic and logical operations are performed using a stack machine, independent of any specific hardware.

5.8.2 Teaching Goals (Instructor)

- Explain why a VM abstraction is useful in a compilation pipeline
- Introduce the stack-based computation model
- Present the VM arithmetic-logical command set
- Present the VM memory access commands (**push** and **pop**) and memory segments
- Visualize VM execution step by step

5.8.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain what a virtual machine is and its role in compilation
- Describe stack-based computation and stack discipline
- Explain VM arithmetic-logical and comparison commands
- Explain VM **push** and **pop** commands and how segments are mapped to RAM
- Trace VM execution step by step using visualization tools

5.8.4 Key Concepts

1. Why a Virtual Machine?

- High-level languages should not depend directly on a specific instruction set
- A VM provides a uniform, hardware-independent target for compilers
- VM code can be translated to a concrete hardware platform using a VM translator

VM = a contract between compilers and machines

2. Stack-Based Computation

- Operands are pushed onto a stack
- Operations **pop** operands and **push** results
- This model simplifies expression evaluation and intermediate results

3. VM Arithmetic, Logical, and Comparison Commands

- Arithmetic: `add`, `sub`, `neg`
- Logical: `and`, `or`, `not`
- Comparison: `eq`, `lt`, `gt`

Each operation:

- Consumes values from the stack
- Produces a new value on the stack

4. Push and Pop (Memory Access)

- `push` copies a value onto the stack
- `pop` removes a value from the stack

Conceptually:

- Stack = temporary workspace
- Memory = longer-term storage

At this level, memory is treated abstractly, not as RAM addresses.

5. Stack Discipline

- Stack has a **top**
- Operations must respect order:
 - Last-in → first-out
- Incorrect order leads to incorrect results

Stack discipline is the core rule of the VM.

6. Abstraction and Portability

- VM code hides machine-level details from compilers
- The VM translator connects the abstract VM to a concrete platform

5.8.5 Project 7: VM Translator (Part I)

Project Goal

Understand VM behavior by simulating stack-based execution, not by translating VM code to assembly.

What Students Do

Using **Python Tutor**, students:

- Write small programs that simulate:
 - A VM stack
 - Push and pop operations
 - Arithmetic and comparison commands
- Step through execution visually
- Observe stack changes line by line

Required Tasks

i. Task 1: Stack Arithmetic

- Simulate:
 - push constant
 - add, sub, neg
- Trace stack contents after each step

ii. Task 2: Logical & Comparison Operations

- Simulate:
 - and, or, not
 - eq, lt, gt
- Observe boolean results on the stack

iii. Task 3: Explanation

- Written explanation of:
 - Stack state changes
 - Order of operations
 - Why stack discipline matters

What Is NOT Required

- Writing a VM translator
- Generating assembly code
- Managing RAM addresses or SP registers

Tools Used

- **Python Tutor**

Why Python Tutor:

-
- Visualizes stacks clearly
 - Step-by-step execution
 - Language-agnostic concepts
 - Perfect for VM abstraction

5.8.6 Common Student Difficulties

- Forgetting stack order
- Confusing stack top with bottom
- Expecting random access to stack values
- Skipping step-by-step tracing

Instructor tip: Make students draw the stack after *every* operation

5.8.7 End-of-Lecture Summary

Students now understand:

- Why virtual machines exist
- How stack-based computation works
- How VM commands manipulate the stack
- How abstraction enables portability

This completes the transition from [machine-level execution](#) → [virtual execution](#)

5.8.8 Preparation for Next Lecture

- Review VM command syntax and segment mapping rules
- Read Chapter 8, *Virtual Machine II: Program Control*
- Continue Project 7 incrementally, testing in the order of the supplied test programs

5.9 Week 10 – Lecture 8 (Chapter 8): Virtual Machine II: Program Control

5.9.1 Lecture Purpose

Extend the virtual machine abstraction to support program control, including branching, loops, and function calls. This lecture explains how structured programs execute using a call stack and activation records, independent of any specific hardware or instruction set.

5.9.2 Teaching Goals (Instructor)

- Explain VM-level branching and program flow commands (*commands that control jumps, loops, and conditional execution, such as `label`, `goto`, and `if-goto`*)

-
- Explain why function calls require a call stack and a calling protocol
 - Introduce function commands and activation records (call frames) (*how function, call, and return work, and how each function call creates its own stack frame to store local variables and saved segment pointers*)
 - Show how return addresses and segment pointers are saved and restored
 - Reinforce abstraction and portability

5.9.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain how branching and loops work in a VM model
- Describe how function calls and returns are executed
- Explain the structure and purpose of the call stack
- Understand how local variables and arguments are isolated per function
- Trace nested and recursive calls step by step

(Students do **not** implement a VM translator.)

5.9.4 Key Concepts

1. VM Program Control Commands

- Branching commands: `label`, `goto`, `if-goto`
- Support conditional and unconditional control flow
- Labels must be translated in a way that avoids naming collisions

2. Branching and Conditional Execution

- Branching allows programs to:
 - Skip instructions
 - Repeat instructions
- Conditional branching depends on values produced by computation

Control flow is what makes programs *dynamic*.

3. Why Function Calls Are Needed

- Functions allow:
 - Code reuse
 - Modular program structure
- Each function call must:
 - Receive arguments
 - Have its own local variables
 - Return control to the caller

4. Call Stack

- The **call stack** keeps track of:
 - Active function calls
 - Return points
 - Local execution context
- Each function call creates a new frame

The stack remembers *where to return* and *what state to restore*.

5. Function Frames (Activation Records)

A function frame typically stores:

- Return address
- Local variables
- Arguments
- Saved execution state

Each function has its own frame, even when called multiple times.

6. Function Call and Return

- **Call:**
 - Create a new frame
 - Transfer control to the function
- **Return:**
 - Produce a return value
 - Destroy the current frame
 - Resume execution in the caller

This supports nested and recursive calls.

7. Abstraction and Portability

- VM programs do not depend on:
 - Registers
 - Instruction sets
 - Memory layouts
- The same VM model works across platforms

5.9.5 Project 8: VM Translator (Part II)

Project Goal

Understand program control and function calls by simulating VM execution, not by translating code.

What Students Do

Using **Python Tutor**, students:

- Trace programs that include:
 - Branches
 - Loops
 - Function calls
- Observe:
 - Call stack growth and shrinkage
 - Local variable isolation
 - Return behavior

Required Tasks

i. Task 1: Branching & Loops

- Simulate a loop-based program
- Identify how control flow changes execution order

ii. Task 2: Function Calls

- Trace a program with:
 - One function call
 - Nested calls
- Observe stack frames

iii. Task 3: Recursion (Conceptual)

- Trace a simple recursive function
- Observe:
 - Multiple frames
 - Return order

What Is NOT Required

- Writing a VM translator
- Generating assembly code
- Managing bootstrap logic
- Handling multi-file programs

Tools Used

- **Python Tutor**

Why Python Tutor:

- Visual call stack
- Clear frame separation
- Step-by-step control flow
- Perfect for VM-level reasoning

5.9.6 Common Student Difficulties

- Confusing stack frames with the data stack
- Confusing the data stack (used for expressions) with the call stack (used for function calls)
- Forgetting where control returns after a function
- Misunderstanding recursion
- Losing track of execution order

Instructor tip: Make students predict the call stack before stepping.

5.9.7 End-of-Lecture Summary

Students now understand:

- How program control works in a VM
- Why function calls need a stack
- How frames isolate execution state
- How abstraction enables structured programming

This completes the VM abstraction layer.

5.9.8 Preparation for Next Lecture

- Review stack and call stack concepts
- Practice tracing nested calls
- Read *Chapter 9: High-Level Language*

5.10 Week 11 – Lecture 9 (Chapter 9): High-Level Language

5.10.1 Lecture Purpose

Introduce high-level programming and object-oriented programming (OOP) as a way to design software using abstraction, objects, state, and behavior, without dealing with hardware, virtual machines, or low-level execution details.

This lecture shifts focus from how programs execute to how programs are designed and structured.

5.10.2 Teaching Goals (Instructor)

- Explain why high-level languages are needed
- Introduce block-based high-level programming
- Present core programming constructs:
 - Variables
 - Expressions
 - Control flow (conditionals and loops)
- Explicitly introduce the four core OOP concepts
- Show how OOP supports modular program design
- Enable students to build a small interactive program

5.10.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain why high-level languages exist
- Use variables to represent program state
- Implement conditionals and loops
- Explain the four pillars of OOP:
 - Encapsulation
 - Abstraction
 - Inheritance
 - Polymorphism
- Build a small interactive program using objects

5.10.4 Key Concepts

1. Why High-Level Languages?

- Low-level programming focuses on hardware details
- Low-level code is:
 - Hard to read
 - Hard to maintain
- High-level languages:
 - Increase productivity
 - Reduce errors
 - Hide execution details

Key idea: High-level programming focuses on designing behavior, not managing machines.

2. Program Structure

-
- Programs are composed of:
 - Objects
 - State
 - Behavior
 - In Snap!:
 - Sprites represent objects
 - Sprite variables represent state
 - Scripts and custom blocks represent behavior

Programs are collections of interacting objects.

3. Variables and State

- Variables store program state
- State can be:
 - Object-local
 - Shared (global)
- Well-designed programs minimize shared state
- Objects are responsible for managing their own data

4. Control Flow

- Programs use:
 - Conditionals (`if / else`)
 - Loops (`repeat / forever`)
- Control flow determines:
 - When behavior runs
 - How often behavior runs

Control flow gives programs dynamic behavior.

5. Object-Oriented Programming

Object-oriented programming organizes programs around [objects](#).

Each object has:

- **State** – what it remembers
- **Behavior** – what it can do

OOP improves:

- Modularity
- Reuse
- Maintainability

6. The Four Pillars of Object-Oriented Programming

i. Encapsulation

- Objects manage their own state and behavior
- Internal data is not directly manipulated by other objects

In Snap!:

- Sprite-local variables enforce encapsulation

ii. Abstraction

- Objects expose what they do, not how they do it
- Users interact with objects through methods

In Snap!:

- Custom blocks act as methods

Abstraction hides complexity.

iii. Inheritance (Conceptual: Behavior Reuse)

- Inheritance is introduced conceptually, not as a class-based mechanism
- The goal is to understand reuse and extension of behavior, not class hierarchies
- New objects are created by copying existing objects and modifying behavior

In Snap!:

- Sprites are duplicated to reuse existing variables and blocks
- The duplicated sprite is treated as a derived version
- Changes to the duplicated sprite do not affect the original

This models the *idea* of inheritance (reuse + extension) without implementing real class-based inheritance.

iv. Polymorphism

- Different objects respond differently to the same method call
- Same message, different behavior

In Snap!:

- Multiple sprites implement blocks with the same name but different logic

7. Relationship to Execution (Conceptual Only)

-
- High-level operations eventually become:
 - Control flow
 - State updates
 - Students do not see the VM or hardware
 - Prior execution knowledge explains why high-level programs work

5.10.5 Project 9: High-Level OOP Design Exercise

Project Goal

Develop a conceptual understanding of high-level programming and object-oriented design by constructing a small interactive object system.

The emphasis of this project is on object structure, behavior, and abstraction, not on software complexity, performance, or completeness.

What Students Do

Using Snap!, students:

- Build a small interactive program
- Create at least two objects
- Assign state and behavior to each object
- Demonstrate all four OOP pillar

Required Tasks

i. Task 1: Encapsulation

Objective: Show that each object manages its own internal state.

Students must:

- Create at least one object-local variable inside an object (sprite)
- Ensure the variable is not global
- Demonstrate that each object instance maintains its own independent state

ii. Task 2: Abstraction

Objective: Separate *what an object does* from *how it does it*.

Students must:

- Create at least one custom block (method)
- Use the custom block to control or modify the object's behavior
- Invoke the block without exposing its internal logic

iii. Task 3: Inheritance (Conceptual)

Objective: Demonstrate reuse of structure and behavior.

Students must:

- Duplicate an existing object (sprite)
- Treat the duplicated object as a derived object
- Reuse the original object's variables and/or blocks
- Modify at least one aspect of the duplicated object's behavior

Note: Snap! does not implement formal class-based inheritance. This task models conceptual inheritance through behavior reuse and extension.

iv. Task 4: Polymorphism

Objective: Demonstrate that the same message can produce different behavior depending on the object.

Students must:

- Define a custom block with the same name in at least two different objects
- Implement different internal behavior for that block in each object
- Execute the block on each object and observe different results

Required Features

Each submission must include:

- At least two objects (sprites)
- At least one object-local variable per object
- At least one custom block per object
- Use of:
 - Conditional logic (`if`)
 - Repetition (`repeat`, `forever`, or similar loop)

What Is NOT Required

- Building a full game
- Scoring systems or levels
- Text-based programming
- Compiler or VM translation
- Low-level execution tracing

Tools Used

- Snap!

Why Snap!:

- Visual objects and state
- No syntax errors
- Clear method abstraction
- Ideal for learning OOP concepts

5.10.6 Common Student Difficulties

- Confusing object state with global state
- Treating blocks as commands instead of behaviors
- Overusing global variables
- Attempting to build everything at once

Instructor tip: Have students explicitly identify each OOP pillar in their own program.

5.10.7 End-of-Lecture Summary

Students now understand:

- Why high-level languages exist
- How programs are structured using objects
- The four pillars of object-oriented programming
- How abstraction enables scalable software design

This completes the transition from execution → design.

5.10.8 Preparation for Next Lecture

- Review objects, methods, and state
- Reflect on how high-level programs execute conceptually
- Read Chapter 10: *Program Structure & Syntax*

5.11 Week 12 – Lecture 10 (Chapter 10): Compiler I: Syntax & Structure

5.11.1 Lecture Purpose

Introduce the front end of a compiler conceptually, focusing on how high-level programs are structured and analyzed before execution. Students learn how programs are checked for structural correctness before they are allowed to run, without implementing a real compiler. This lecture explains why structure must be verified before code generation.

5.11.2 Teaching Goals (Instructor)

-
- Explain what a compiler is and why it exists
 - Introduce the stages of compilation conceptually
 - Explain syntax and program structure
 - Show how language rules define valid programs
 - Demonstrate how syntax analysis precedes execution and code generation

5.11.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain what a compiler does at a high level
- Describe how programs are analyzed **before execution**
- Understand the idea of tokens, statements, and structure
- Explain how syntax rules constrain program form
- Relate program structure to execution flow

(Students do not build a tokenizer or parser.)

5.11.4 Key Concepts

1. What Is a Compiler?

- A compiler translates human-written programs into executable behavior
- This happens in stages, not all at once
- Early stages focus on structure, not execution

Key idea: A compiler must understand the structure of a program before it can generate code.

2. Compilation Pipeline (Conceptual)

A simplified view:

High-level program
→ structured representation
→ executable behavior

This lecture focuses on:

- Understanding program structure
- Ensuring programs follow language rules

Later lectures build on this structure to explain [code generation](#).

3. Syntax and Structure

-
- **Syntax** = rules for how programs are written
 - Programs are made of:
 - Statements
 - Expressions
 - Blocks

Invalid structure → program cannot run.

In Snap!:

- Blocks only fit together in valid ways
- Structure is enforced before execution

Snap! acts like a built-in syntax checker.

4. Tokens (Conceptual)

In text-based languages, compilers break programs into simple building blocks called tokens (such as words, numbers, and symbols).

In Snap!:

- Blocks already represent structured syntax
- Tokens are implicit and visual
- Syntax errors are prevented rather than reported

Students experience the *result* of tokenization without performing it.

5. Grammar Rules

- A grammar defines:
 - What is allowed
 - What is not allowed

Grammar rules determine:

- Where loops can appear
- Where conditions are valid
- How expressions are nested

In Snap!:

- Block shapes visually enforce grammar rules
- Invalid grammar structures cannot be built

This mirrors how a syntax analyzer enforces grammar rules in text-based languages.

6. Hierarchical Program Structure

Programs have nested structure:

- Programs contain blocks
- Blocks contain statements
- Statements contain expressions

This structure is often represented as a tree.

Understanding this hierarchy is essential for code generation.

7. Structure Before Execution

- Compilers analyze structure before running code
- Execution assumes structure is already correct
- Structural errors must be fixed first

Using [Python Tutor](#), students observe how structured programs execute step by step.

This shows why syntax analysis must happen before execution.

5.11.5 Project 10: Program Structure Analysis

Project Goal

Understand how program structure and syntax are analyzed before execution by manually performing the same structural analysis that a syntax analyzer would perform automatically.

What Students Do

Using [Snap!](#), students:

- Build programs with:
 - Conditionals
 - Loops
 - Nested blocks

Using [Python Tutor](#), students:

- Step through equivalent structured programs
- Observe how structure affects execution order

Required Tasks

i. Task 1: Structured Program

- Create a Snap! program with:
 - At least one loop
 - At least one conditional
 - Nested structure

This ensures the program has a non-trivial syntax structure.

ii. Task 2: Structure Mapping (Syntax Analysis Experience)

- Produce one of the following:
 - A block-containment diagram, or
 - A numbered outline showing nesting and execution order

In this task, students are manually performing syntax analysis by identifying program structure.

iii. Task 3: Execution Connection

- Use Python Tutor to trace a similar structured program
- Explain how structure determines execution order

This reinforces that execution depends on prior structural correctness.

What Is NOT Required

- Writing a tokenizer or parser
- Producing XML
- Implementing grammar rules in code

5.11.6 Common Student Difficulties

- Confusing structure with execution
- Writing deeply nested code without understanding flow
- Assuming order does not matter
- Treating blocks as independent instead of hierarchical

Instructor tip: Ask students: “*What block contains this statement?*”

5.11.7 End-of-Lecture Summary

Students now understand:

- What a compiler does before execution

-
- Why syntax and structure matter
 - How programs are organized hierarchically
 - Why code generation requires a correct structure first

This prepares students directly for execution and translation concepts.

5.11.8 Preparation for Next Lecture

- Review how structured programs execute step by step
- Reflect on how Snap! enforces syntax visually
- Read *Chapter 11: Execution & Translation*

5.12 Week 13 – Lecture 11 (Chapter 11): Compiler II – Execution & Translation (Conceptual)

5.12.1 Lecture Purpose

Explain how high-level programs are translated into executable behavior. This lecture focuses on meaning (semantics) rather than syntax, showing how variables, expressions, control flow, and function calls are lowered step by step into concrete execution actions. Students learn what must happen internally for a correct program to run, without implementing a compiler or generating intermediate code.

5.12.2 Teaching Goals (Instructor)

- Explain what code generation means at a conceptual level
- Show how high-level constructs are broken into simple execution steps
- Explain why names, scope, and memory locations matter at runtime
- Connect program structure to actual runtime behavior
- Reinforce the idea of lowering abstraction levels

5.12.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain how high-level programs become executable
- Describe how variables are mapped to memory locations
- Explain how expressions are evaluated step by step
- Understand how control flow is implemented at runtime
- Trace function calls and returns during execution

(Students do not generate VM code, assembly code, or compiler components.)

5.12.4 Key Concepts

1. From Structure to Behavior

- [Syntax](#) defines what programs are allowed to look like
- [Code generation](#) defines how programs behave when they run
- Each high-level construct expands into many small execution steps

Key idea:

Big ideas are executed as many small actions.

2. Meaning Beyond Syntax (Semantics)

Programs carry meaning such as:

- What a variable refers to
- Where that value is stored
- When it is updated

This information must be resolved before and during execution.

Syntax alone is not enough to run a program.

3. Variables, Names, and Memory

- Variable names do not exist at runtime
- Names are replaced by memory locations
- Scope determines:
 - Which variable is used
 - When it exists

In this course, symbol information is handled automatically by the system.

4. Expression Evaluation

- Expressions are evaluated:
 - One operation at a time
 - In a defined order
- Intermediate results are stored temporarily

Using [Python Tutor](#), students observe:

- Temporary values

-
- Evaluation order
 - Stack usage

This reveals how even simple expressions require multiple steps.

5. Control Flow at Runtime

- High-level constructs such as `if` and `while` become:
 - Tests
 - Jumps
 - Repeated execution

Control flow reshapes the execution path based on conditions and loops.

6. Functions and Execution Context

- Each function call:
 - Creates a new execution context
 - Has its own variables
- Context is created on call and destroyed on return

This explains:

- Nested calls
- Recursion
- Call stack behavior

7. Abstraction Lowering

High-level construct

→ simpler execution actions

→ runtime behavior

This process is the essence of code generation, even when no VM or assembly code is shown.

5.12.5 Project 11: Execution Tracing & Translation Concepts

Project Goal

Understand how a correct, structured program is executed at runtime by tracing its execution and explaining how high-level constructs (variables, expressions, control flow, and functions) are lowered into concrete execution steps.

This project reinforces Lecture 11 by making program execution visible, emphasizing meaning (semantics) rather than syntax or implementation.

What Students Do

Using **Snap!**, students:

- Build a program with:
 - Variables
 - Expressions
 - Conditionals
 - Loops
 - Custom blocks (functions)

Snap! is used to represent what the program is meant to do, not how it is executed internally.

Using **Python Tutor**, students:

- Trace an equivalent program step by step
- Observe:
 - Variable creation and updates
 - Expression evaluation
 - Function calls and returns
 - Call stack behavior

Python Tutor is used to reveal runtime behavior that is normally hidden.

Tools Used

- **Snap!** — to represent high-level program intent
- **Python Tutor** — to observe step-by-step execution and runtime state

(Students do not translate Snap! directly into Python; they write a simple equivalent program.)

Required Tasks

i. Task 1: Program Construction

- Build a Snap! program with:
 - At least one custom block (function)
 - At least one conditional
 - At least one loop
 - At least one variable that is updated during execution

The program should be small but non-trivial, so that execution involves multiple steps and changing state.

ii. Task 2: Execution Trace (Runtime Behaviour)

- Use Python Tutor to trace a similar program

-
- Students must record:
 - The order of execution
 - How variable values change over time
 - When functions are called and returned
 - How the call stack grows and shrinks

Students are observing execution, not debugging.

iii. Task 3: Translation Explanation (Conceptual)

- Write a short explanation (5–8 sentences) describing:
 - How high-level statements were executed as multiple small runtime steps
 - How variables were created, updated, and stored
 - How expressions were evaluated in a specific order
 - How function calls created and destroyed execution contexts

Students explain what happened internally, not how to implement a compiler.

What Is NOT Required

- Writing a compiler
- Generate VM or assembly code
- Implement symbol tables
- Managing symbol tables explicitly
- Running emulators or low-level tools

5.12.6 Common Student Difficulties

- Assuming variables exist “by name” at runtime
- Confusing scope with lifetime
- Losing track of intermediate values
- Misunderstanding call stack behavior

Instructor tip: Ask students: “*Where is this value stored right now?*”

5.12.7 End-of-Lecture Summary

Students now understand:

- How high-level programs become executable
- Why variables and scope matter at runtime
- How expressions and control flow execute
- How function calls create runtime behavior

This completes the conceptual compiler pipeline: [Design](#) → [Structure](#) → [Execution](#)

5.12.8 Preparation for Next Lecture

- Review how programs depend on runtime services
- Reflect on how execution requires support beyond the program itself
- Read *Chapter 12: Operating System (Conceptual)*

5.13 Week 14 – Lecture 12 (Chapter 12): Operating System (Conceptual)

5.13.1 Lecture Purpose

Introduce the operating system (OS) as a layer of software services that sits between applications and hardware. This lecture emphasizes that an OS is not hardware and not magic, but a program that provides reusable abstractions and services, completing the system built throughout the course.

5.13.2 Teaching Goals (Instructor)

- Explain what an operating system is at a conceptual level
- Identify the core services provided by an OS
- Show that OS functionality can be implemented as software libraries
- Connect OS services to:
 - High-level programs
 - Execution behavior
 - Underlying hardware
- Reinforce abstraction as the unifying theme of the course

5.13.3 Student Learning Outcomes

By the end of this lecture, students will be able to:

- Explain the purpose and role of an operating system
- Identify common OS services (memory, I/O, timing)
- Explain how OS abstractions simplify application development
- Describe how applications interact with hardware indirectly
- Understand the OS as the final abstraction layer in the system stack

5.13.4 Key Concepts

1. What Is an Operating System?

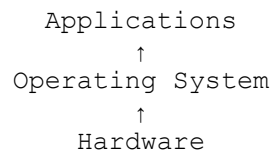
An Operating System (OS) is system software that sits between applications and hardware.

The OS controls the computer and provides services so applications do not deal with hardware directly.

Key ideas:

- Applications do not access hardware directly
- The OS acts as an intermediary and manager

2. OS Position in the System Stack



Each layer:

- Uses services of the layer below
- Hides complexity from the layer above

Core theme: abstraction and control.

3. Why Applications Need an OS

Without an OS:

- Programs manage hardware themselves
- Code is complex, fragile, and unsafe

With an OS:

- Programs are simpler
- Resources are shared safely
- Hardware details are hidden

4. Core Functions of an OS

i. Process (CPU) Management

- Many programs share one CPU
- The OS decides which program runs and when
- Programs appear to run in parallel

Key terms (conceptual):

- Process
- Scheduling
- Context switching

ii. Memory Management

- Memory is shared and limited
- Each program gets its own logical memory space
- Programs cannot access each other's memory

Key ideas:

- Isolation
- Safety

iii. File Management

- Data is organized as files and directories
- OS provides create, read, write, delete operations

Files are an abstraction over raw storage.

iv. Input / Output (I/O) Management

- Devices: keyboard, screen, disk, network
- OS hides device details
- Provides uniform access

Applications use OS services, not devices directly.

5. OS as a Service Provider

The OS provides services such as:

- Running programs
- Input and output
- File access
- Memory usage
- Timing and waiting

Applications request services; they do not control hardware.

6. System Calls (Conceptual)

A system call is how an application requests an OS service.

Conceptual flow:

Application → OS → Hardware → OS → Application

Key rule:

-
- Applications cannot bypass the OS

7. OS as Software (Educational Model)

In this course:

- The OS is treated as a normal program
- OS services are library-like functions

Not covered:

- Kernel details
- Interrupts
- Scheduling algorithms
- Hardware privilege levels

Focus: what the OS does, not how real OS kernels are built.

8. Abstraction and Protection

- **Abstraction:** hides hardware and complexity
- **Protection:** prevents programs from interfering with each other

This enables stable and secure systems.

9. Multi-Program Environment

- Many programs appear to run at once
- OS rapidly switches between them

The OS creates the illusion of parallelism.

10. OS as a Resource Manager

The OS manages:

- CPU time
- Memory
- Storage
- Devices

It fairly allocates limited resources.

11. Big Picture Connection

- Hardware executes instructions

-
- Programs describe behavior
 - OS manages execution and resources

Program → System Call → OS → Hardware

This completes the course journey from logic gates to applications.

5.13.5 Project 12: Operating System Services Simulation

Project Goal

Design a conceptual operating system that provides basic services to applications, demonstrating how OS abstractions separate applications from hardware. This project completes the course by showing how all programs rely on a shared software layer rather than direct hardware access.

What Students Do

Using **Snap!**, students create two clearly separated components:

1. *OS Sprite (Operating System)*

It defines custom blocks (services) such as:

- `print(text)` — display output
- `wait(seconds)` — pause execution
- `getInput()` — receive user input
- `allocateMemory()` — conceptual memory service
- `drawOnScreen()` — display graphics

The OS sprite:

- hides all implementation details
- may use Snap!’s built-in blocks internally
- exposes only clean service interfaces

2. *Application Sprite*

- Uses OS services exclusively
- Never accesses the stage or hardware-like features directly
- Calls OS blocks to perform tasks

Tool Used

- **Snap!**

Why Snap!

- Makes OS ideas visible

-
- Models services and interfaces clearly
 - Avoids low-level complexity
 - Reinforces abstraction over implementation

Required Tasks

- Create at least three OS services
- Create at least one application
- Demonstrate:
 - Application → OS interaction
 - OS hiding implementation details

What Is NOT Required

- Kernel implementation
- Interrupt handling
- Memory paging or scheduling
- Real OS concurrency
- Jack OS classes
- Hardware privilege mechanisms

Project Focus

- Clear separation between application and OS
- Correct use of abstraction
- Clean service interfaces
- Conceptual correctness

5.13.6 Common Student Difficulties

- Thinking the OS is hardware
- Expecting applications to control devices directly
- Overcomplicating OS services

Instructor tip: Ask students: *“What service is the OS providing here?”*

5.13.7 End-of-Lecture Summary

Students now understand:

- What an operating system really is
- Why programs rely on OS services
- How abstraction enables large systems
- How the entire system stack fits together

This completes the journey from logic gates → applications.

5.13.8 Preparation for Final Assessment

- Review all abstraction layers studied in the course
- Revisit how each layer depends on the one below
- Prepare for integration-focused questions
- Reflect on how simple components build powerful systems

6 Licensing and Usage Notes

Table 4 provides an overview of third-party educational tools and materials used in this course, together with their official licenses and classroom usage permissions.

Inclusion of this table is intended to support transparent, lawful, and ethical use of all external resources within the course.

Table 4: Licensing and Classroom Usage Conditions for Third-Party Tools and Materials

Tool Resource /	What it is	License / copyright (publisher statement)	Official license/copyright page	Classroom teaching + semester “reading/using” (teachers & students)
Digital (hneemann/Digital)	Logic circuit simulator	GNU GPL v3.0 (strong copyleft) — use/modify allowed, but distributing the program (or modified versions) requires providing the complete corresponding source under GPL v3 and keeping copyright/license notices.	https://github.com/hneemann/Digital/blob/master/LICENSE	OK for classroom use (install/run for labs). If you distribute copies (zip/installer/modified build), you must also provide the corresponding source code under GPL v3 and keep notices.
Snap!	Block-based programming environment	GNU AGPL v3.0 for the Snap! software (strong copyleft for network use). Snap! site also provides a DMCA policy.	https://github.com/jmoenig/Snap/blob/master/LICENSE ; https://snap.berkeley.edu/dmca	OK for classroom use. If you host a modified Snap! instance for students, AGPL may require offering the complete corresponding source of your modified version to users accessing it over the network. Using the official site/unmodified version is fine.
Python Tutor (OnlinePython Tutor codebase)	Program execution visualizer (web app)	GNU GPL v3 (or later) — the software is free software under GPL v3+; Copyright (C) 2010 Philip J. Guo.	https://github.com/hcietist/OnlinePythonTutor/blob/master/README	OK for classroom use. Students can use the public site or you can run it internally. If you distribute copies (VM image/modified build), comply with GPL (keep notices + provide corresponding source).
NandGame	“Build a computer from scratch” browser game	No license stated on the official About page → treat as standard copyright by default.	https://www.nandgame.com/about	OK to use by linking for lectures/labs/homework during the semester. Avoid rehosting/redistributing unless permission is given.
Little Man Computer (Peter Higginson)	Little Man Computer CPU simulator (browser-based)	Copyright notice shown: © 2014–23 Peter L Higginson.	https://peterhigginson.co.uk/lmc/help.html	OK for semester teaching by linking. Do not use automated/synchronized loads (don’t force-open on all machines at once).
RISC Simulator (Peter Higginson)	RISC CPU teaching simulator (browser-based)	Copyright notice shown: © 2016–23 Peter L Higginson. Includes explicit ‘Use in Classrooms’ conditions prohibiting automated or synchronized loading that may overload the hosting service	https://peterhigginson.co.uk/RISC/help.html	OK for semester teaching by linking. Allowed: 100 students open manually over a few minutes. Not allowed: classroom system auto-opens for everyone at once (same second/few seconds) or mass-refreshing.
nand2tetris (course materials & tools)	Course + tools (“From Nand to Tetris”)	CC BY-NC-SA 3.0 Unported for all materials/tools. They also request not posting solutions publicly, but private/password-protected sharing is OK.	https://www.nand2tetris.org/license	OK for semester teaching and student use (non-commercial, attribution required, ShareAlike for adaptations). Use private LMS/private repo for sharing work; avoid publishing solutions publicly.
<i>The Elements of Computing Systems</i> (MIT Press, 2nd ed., 2021)	Textbook (nand2tetris book)	© 2021 Massachusetts Institute of Technology. All rights reserved. No reproduction without written permission from the publisher.	https://mitpress.mit.edu/9780262539807/the-elements-of-computing-systems/	OK to assign/read during the semester via legal access (purchase/library/licensed ebook). Do not upload/share full scans/PDF chapters unless permitted by license or fair use/course reserves policy.

7 References

- [1] N. Helmut, “Digital: A digital logic designer and circuit simulator,” Github. Accessed: Jan. 21, 2026. [Online]. Available: <https://github.com/hneemann/Digital?tab=readme-ov-file>
- [2] L. H. Peter, “Little Man Computer - CPU simulator .” Accessed: Jan. 21, 2026. [Online]. Available: <https://peterhigginson.co.uk/lmc/help.html>
- [3] P. J. Guo, “Online python tutor,” pp. 579–584, Mar. 2013, doi: 10.1145/2445196.2445368.
- [4] “ Snap! Build Your Own Blocks.” Accessed: Jan. 21, 2026. [Online]. Available: <https://snap.berkeley.edu/versions>
- [5] O. J. Kjær, “NandGame - Build a computer from scratch.” Accessed: Jan. 21, 2026. [Online]. Available: <https://nandgame.com/about>
- [6] L. H. Peter, “RISC Simulator.” Accessed: Jan. 21, 2026. [Online]. Available: <https://peterhigginson.co.uk/RISC/>